

Clutter

500-MM-3-Clutter

Project Manager

Santiago Garza Fuentes
UCID: 30148120
santiago.fuentes@ucalgary.ca

Team Members

Hamza Amar
UCID: 30144227
hamza.amar@ucalgary.ca

Patel Maitra
UCID: 30143020
maitra.patel@ucalgary.ca

Wiedasa Nimna
UCID: 30146042
nimna.wiedasa@ucalgary.ca

Syril Jacob
UCID: 30145947
syril.jacob@ucalgary.ca

Singh Akshpreet
UCID: 30129354
akshpreet.singh1@ucalgary.ca

Academic Advisor

Lorenzo De Carli
lorenzo.decarli@ucalgary.ca

Alberta 2514640 Inc

Devin Atkin
devin.atkin@alberta2514640.ca

April 12, 2026

Contents

1	Glossary	iii
	Terms	iii
	Acronyms	iv
2	Executive Summary	1
3	Project Motivation and Objectives	2
	Motivation	2
	Problem Statement	2
	Objectives	3
4	Methodology and Design	4
	System Overview	4
	System Architecture	4
	Frontend Design	4
	Backend Design and API layer	5
	Data Model and Storage	6
	Database Schema	6
	S3 Storage	7
	Terraform Generation	7
	Engine	7
	Deployment Pipeline	7
	Ansible EC2 Configuration Pipeline	8
	Security and Authentication	8
	End-to-End Workflow	9
5	Project Scope and Final Product Functionality	13
	Scope	13
	Deviation of the Final Scope	14
	Product Functionality	15
	Client Sign On	15
	Organization Management	15
	Project Management	16

Diagram Management	18
AWS Account Linking Process	19
Diagramming Interface	21
Terraform Engine	23
Terraform Deployer	25
Ansible Runner	27
6 Technical Specifications	28
Local Dev Environment	32
Access Control (IAM)	33
Build and CI Time	33
7 Measures of Success and Validation Test Results	34
Functional Validation Against Sponsor Expectations	34
Technical Validation	35
API and Backend correctness	35
Terraform Generation and Deployment	35
Usability Testing for Non-Technical Users	36
8 Tools, Materials, Supplies, and Cost	39
9 Reflections	41
Ethics and Equity in our Capstone Journey	41
Impact of Engineering on Society and Environment	41
Team’s Approach to Project Management	42
Team’s Approach to Economics and Cost Control	43
Team Members’ Approach to Life-Long Learning	44
A Additional Material	46
AI Use Statement	46
Full MIT License Text	46

Glossary

Terms

Clutter Drag-and-drop canvas that generates Terraform/Ansible artifacts to deploy AWS infrastructure.

Workspace Execution context of a project to AWS account/region/backends for plan/apply runs.

React (Next.js 14)

Front-end framework used to build the SPA, including routing, forms, and the diagram canvas.

Terraform Infrastructure-as-Code tool that materializes the diagram into AWS resources (plans/modules/vars).

Ansible Configuration management used to set up software on EC2/container targets after infrastructure creation.

Docker Containerization used for reproducible build/run environments (e.g., ephemeral Terraform runner).

DynamoDB NoSQL store holding tenants, projects, canvases, nodes, edges, runs, etc., optimized for specific query patterns.

Global Secondary Index (GSI)

DynamoDB index that enables alternate access paths.

AWS Lambda Serverless compute for stateless API handlers that back the project, canvas, and run endpoints.

API Gateway HTTP front door that routes authenticated client requests to Lambda APIs.

AWS S3 Object storage for generated bundles, logs, and run artifacts.

AWS IAM Roles and policies implementing least-privilege access across Lambda, S3, DynamoDB, and STS.

JWT JSON Web Token used to authenticate users and authorize tenant-scoped API access.

Acronyms

UI	User Interface.
UX	User Experience.
API	Application Programming Interface.
IaC	Infrastructure as Code.
SPA	Single-Page Application.
CRUD	Create, Read, Update, Delete.
GSI	Global Secondary Index.
VPC	Virtual Private Cloud.
VM	Virtual Machine
STS	Security Token Service.
CI/CD	Continuous Integration / Continuous Deployment.

Executive Summary

Provisioning cloud infrastructure is a critical part of modern software development, but for smaller teams it often presents a steep learning curve. While Infrastructure as Code tools like Terraform make deployments reusable, modular, and automatable, they also require a deep understanding of configuration syntax, dependencies, and provider-specific settings. For teams with limited cloud experience, writing and maintaining these configurations can quickly become time-consuming and error-prone. This slows projects down and increases business risk.

Clutter (Cloud Layout and Utilization Tool for Topology and Environment Rollouts) was built to solve this problem. Developed for Alberta 2514640 Inc., Clutter is a web-based, drag-and-drop interface that allows non-expert users to describe a target AWS infrastructure visually and automatically generate the corresponding Terraform and Ansible deployment artifacts, without writing a single line of IaC manually.

The system is built on a fully serverless AWS architecture. The frontend is implemented in Next.js and React, powered by a React Flow, allows users to drag, drop, and connect AWS resource blocks. The backend consists of Go-based Lambda functions exposed through API Gateway, covering approximately 31 endpoints across 10 domains. Diagram state is persisted in a PostgreSQL database, while Terraform artifacts, Ansible playbooks, and deployment logs are stored in S3. Infrastructure deployment is handled by ephemeral AWS Fargate tasks that assume cross-account IAM roles via STS to apply or destroy resources directly in the client's AWS account. EC2 configuration is managed through a serverless Ansible pipeline that uses SSM Run Command.

The final product delivers on all core requirements set by the sponsor. Users can authenticate via Google OAuth, create organizations and projects, build multi-component AWS diagrams, and deploy or destroy infrastructure through the interface. The Terraform engine correctly generates `main.tf` and `outputs.tf` files from diagram data, including automatically derived IAM policies from resource connections. Deployment and destruction of infrastructure was validated against a simulated client AWS account, with live log streaming and persistent S3 log files. Backend correctness was verified through a comprehensive Postman test suite. Usability testing with non-technical users confirmed that the drag-and-drop interface is accessible without prior knowledge of Terraform or cloud infrastructure.

Two features were descoped during the project: (1) the ability to extract and replicate an EC2 environment locally, and (2) collaborative multi-user diagramming. These decisions were communicated to the sponsor, and the team prioritized delivering a solid, fully functional MVP over incomplete implementations of goals.

Project Motivation and Objectives

Motivation

Provisioning cloud infrastructure is a critical part of modern software development, but for smaller teams, it often presents a steep learning curve. While Infrastructure as Code (IaC) tools like Terraform make deployments reusable, modular, and automatable, they also require a deep understanding of configuration syntax, dependencies, and provider-specific nuances. For teams with limited cloud experience, writing and maintaining Terraform configurations can quickly become time-consuming and error-prone. As a result, even relatively small teams end up struggling to get maintainable cloud resources deployed. As a result, even relatively small teams end up struggling to get maintainable cloud resources deployed.

Today, getting the most out of the cloud typically relies on people who already know Terraform, Ansible, or the provider-specific deployment frameworks (CloudFormation for AWS, Azure Resource Manager, etc.) to translate a business or architectural need (“I need a web tier, a database, and an internal network”) into concrete, working infrastructure deployments. When that knowledge is concentrated in a few individuals or teams, you risk creating silos of knowledge. In practice, this means only a small subset of people can design or change infrastructure, which slows projects down and increases business risk.

Due to this, many tools in the market try to abstract cloud compute and networking: they hide provider-specific details and present a simpler interface for specific use cases. However, most of those tools are opinionated toward a single cloud, or they focus on a narrow scenario (e.g., only CI/CD, or only serverless, or only one kind of container deployment). As an example, a small businesses’ owner interested in expanding their digital service, do not have the resources or expertise to manage and deploy a cloud environment or in the cases if they want to hire an external agency to manage this can come with a sticker price shock, leaving the businesses with lack of digital resource and possibly loss of customers.

Problem Statement

In this current environment, non-expert users cannot deploy and provision in cloud/local infrastructure correctly without expert written IaC. Clutter (Cloud Layout and Utilization Tool for Topology and Environment Rollouts) aims to solve this issue by bundling deployment, visualization, and updating AWS cloud compute services into a singular, easy to use diagram drag-drop web GUI interface.

Objectives

The objective of this project is to implement Clutter as a web-based, drag-and-drop interface that allows Alberta 2514640 Inc.'s customers to describe a target infrastructure/topology and automatically generate the corresponding deployment artifacts.

Specifically, the tool will (1) provide a list of pre-planned diagram blocks of the following AWS components (i.e., containerized workloads, DB services and Lambda Functions), (2) translate a finished diagram into ready-to-run Terraform code and Ansible scripts, and finally (3) deploy that design to the client's AWS environment while applying the required routing and networking rules.

A successful outcome is a user being able to create a small, multi-component AWS setup without writing IaC manually, and being able to update it through the diagram with the ability to move some services to a local environment.

Methodology and Design

System Overview

Clutter is a web-based platform that enables users to design AWS infrastructure through a drag-and-drop interface, automatically generate Terraform code from the visual diagram, and deploy that infrastructure directly to a client's AWS account without writing Infrastructure-as-Code manually. The system follows a three-tier layered architecture to ensure separation of concerns, maintainability, and security.

The presentation tier (frontend) provides the diagram editor and dashboard interfaces. The application tier (backend Go Lambda functions) handles business logic, API routing, and orchestration. The data tier (PostgreSQL hosted on Supabase, accessed via the pgx driver, and S3 storage) persists application state, diagram data, Terraform artifacts, and deployment logs. Additional infrastructure includes SQS (with a dead-letter queue) for asynchronous Ansible job queuing, ECR for container image storage, and CloudWatch Logs for Lambda and ECS task observability.

System Architecture

Figure 4.1 illustrates the full system architecture. The frontend communicates through AWS API Gateway to domain-specific Lambda functions. Fargate tasks provide ephemeral compute for Terraform and Ansible execution, SQS queues coordinate asynchronous Ansible job processing, and IAM cross-account roles enable secure deployment into client AWS accounts via STS AssumeRole with `external_id` verification.

Frontend Design

The frontend is built with Next.js 16 and React 19, using TypeScript and Tailwind CSS. The drag-and-drop diagram canvas is powered by React Flow. React Query (TanStack Query) manages all server state, caching, and data synchronization with the backend APIs, including polling-based live log updates during deployments. Framer Motion provides UI animations and transitions. Key frontend components interact with the backend to manage the Google OAuth 2.0 login flow, the project dashboard, and the diagram canvas.

Backend Design and API layer

The backend is composed entirely of AWS Lambda functions written in Go, fronted by API Gateway. Each Lambda handles a specific domain, ensuring clean separation of responsibilities. All endpoints are authenticated via JWT tokens validated by an Authorizer Lambda. The system exposes approximately 31 endpoints across 10 Lambda domains, summarized in Table 4.1.

Table 4.1: Backend Domain Endpoints and Responsibilities

Domain	Endpoint	Description	Auth
Auth	POST /log-in	Google OAuth + JWT issuance	No
Organization	POST/GET /organization	Create and list organizations	Yes
	PUT/DELETE /organization/{id}	Update and delete an organization	Yes
	GET /organization/{id}/accounts	List linked AWS accounts	Yes
	POST /organization/{id}/accounts	Link an AWS account by storing its Role ARN	Yes
	DELETE /organization/{id}/accounts/{aId}	Unlink an AWS account	Yes
Project	POST/GET/PUT/DELETE /project	Full project CRUD	Yes
Diagram	POST/GET/PUT/DELETE /diagram	Full diagram CRUD	Yes
TF Engine	POST /terraform-engine	Generate Terraform code from a diagram	Yes
	POST /.../code-upload/presigned-url	Generate a pre-signed URL for Terraform code upload	Yes
	GET /terraform-engine/logs	Retrieve deployment logs	Yes
	GET /.../logs/url	Get the S3 URL for log retrieval	Yes
	GET /.../logs/live	Provide live deployment log streaming	Yes
	GET /.../logs/recent-activity	Return recent deployment activity	Yes

Domain	Endpoint	Description	Auth
TF Runner	POST /terraform-command-runner	Launch a Fargate task for apply, plan, or destroy operations	Yes
CF Stack	GET /cloudformation-stack-url/{orgId}	Generate a CloudFormation onboarding URL	Yes
Ansible	POST /ansible/jobs	Submit an Ansible job to SQS	Yes
	GET /ansible/jobs	List all submitted jobs	Yes
	GET /ansible/jobs/{jobId}	Retrieve a single job	Yes
	GET /ansible/jobs/{jobId}/logs	Retrieve logs for a specific job	Yes
	POST /ansible/playbooks/upload-url	Generate a pre-signed URL for playbook upload	Yes
User	GET /user-information	Return the user profile data	Yes
Resources	GET /resources	Return supported resource types for the block palette	Yes

Each Lambda function maps to one domain: `/log-in` handles authentication, `/organization` and `/project` manage CRUD and membership, `/diagram` handles canvas persistence with versioned history, `/terraform-engine` generates Terraform code and serves deployment logs, `/terraform-command-runner` launches Fargate tasks for apply/plan/destroy, `/ansible` manages job submission via SQS and playbook uploads, `/resources` serves the block palette, `/cloudformation-stack-url` generates onboarding URLs, and `/user-information` returns profile data.

Data Model and Storage

Database Schema

The application uses PostgreSQL (hosted on Supabase) with 11 tables. The core entities are: `users` (with unique `email`, `full_name`, `picture_url`), `organizations`, and `organization_members` (a many-to-many junction table). Projects belong to organizations, and diagrams belong to projects, storing the full canvas layout as a `jsonb` column (`data`).

The `diagram.history` table maintains versioned snapshots inserted by the update Lambda on every save (using `MAX(version)+1`). The `diagram.deployment_logs` table tracks per-command deployment status using enum types for `command` and `status`. The `aws_account_access_roles` table stores IAM cross-account linking data with `role_arn`, `external_id`, and a status check

constraint (`incomplete/complete/revoked`).

The `playbooks` table links Ansible files to diagrams, organizations, and projects. The `jobs` table records Terraform and Ansible execution with a status lifecycle from `QUEUED` through `STARTING`, `RUNNING`, `COMPLETED`, or `FAILED`. Finally, `supported_resources` is a standalone lookup table queried by the frontend via `/resources` to populate the drag-and-drop block palette, with each row's `variables` JSON defining the configuration fields for that resource type. See Figure 4.2

S3 Storage

The system uses exactly two S3 buckets. The runtime bucket stores all operational data including uploaded playbooks, Ansible logs, Terraform state files, and job outputs, organized under a hierarchical key structure of `orgID/projectID/diagramID/`. The templates bucket is read-only and contains `.tf.tmpl` files for each supported resource type, the CloudFormation YAML for account linking, and bootstrap archive files.

Terraform Generation

Engine

The Terraform generation engine is a Lambda function that ingests diagram JSON data and produces deployable Terraform configuration files (`main.tf` and `outputs.tf`). The engine is triggered via an API call when the user initiates a deployment.

The generation process follows five stages: (1) the API call sends the diagram JSON to the `terraform-engine` Lambda; (2) the engine parses the diagram to extract resource types, variables, and connections between nodes; (3) Go `.tmpl` template files are fetched from an S3 templates bucket; (4) the parsed resources are rendered using Go's `text/template` package to create `main.tf` and `outputs.tf`, with IAM policies automatically generated from diagram edges (for example, a Lambda-to-S3 edge creates the corresponding S3 access policy); and (5) the generated files are uploaded to S3, ready for deployment.

This template-based approach allows new AWS resource types to be supported by adding `.tmpl` files to S3 without modifying the engine's source code.

Deployment Pipeline

The Terraform Deployer runs as an ephemeral AWS Fargate task (task family: `terraform-deployer`). When a user clicks `Apply`, `Plan`, or `Destroy`, the `/terraform-command-runner` Lambda launches a Fargate task that fetches the generated `.tf` files from S3, runs `terraform init`, then performs

STS AssumeRole using the stored `role_arn` and `external_id` to obtain temporary credentials for the client's AWS account. The task then executes the requested Terraform command.

Upon completion, logs and Terraform state are uploaded back to S3, and the command status is written to the `diagram_deployment_logs` table. The frontend polls for status updates via React Query with a 10-second refetch interval.

Client onboarding uses a pre-filled CloudFormation stack. When the user clicks Create Role, the backend generates a CloudFormation URL containing a unique `external_id` and stores a record in `aws_account_access_roles` with status `incomplete`. The user deploys the stack in their AWS account, which creates an `AllowClutterToDeployTerraformRole`.

The user then posts the resulting Role ARN back to Clutter via the `/organization/{orgId}/accounts` endpoint, and the record status is updated to `complete`.

Ansible EC2 Configuration Pipeline

For EC2 instances, Clutter provides Ansible-based configuration using a fully serverless pipeline. The flow proceeds as follows: a POST to `/ansible/jobs` triggers the `ansible-submit-job` Lambda, which enqueues the job into an SQS queue (`ansible_jobs`).

An SQS-triggered Lambda (`ansible-run-task`) launches a Fargate container (`ansible-runner`) that performs STS AssumeRole into the client's account, then uses SSM Run Command to invoke Ansible on the target EC2 instance. Each instance installs Ansible if not present, materializes the playbook from base64, and runs it locally with `--connection=local`.

Execution logs are written to S3 and job status is tracked in the PostgreSQL `jobs` table through a lifecycle from `QUEUED` to `STARTING`, `RUNNING`, `COMPLETED`, or `FAILED`.

Security and Authentication

Authentication uses Google OAuth 2.0. The `/log-in` Lambda validates the Google ID token using `idtoken.Validate()`, creates or fetches the user record, and issues a JWT containing claims for `sub`, `email`, `full_name`, `picture_url`, `created_at`, and `exp`. An Authorizer Lambda validates this JWT on every API call.

Deployment operations use native AWS IAM cross-account roles with STS AssumeRole and `external_id` verification. EC2 configuration uses SSM Run Command (not SSH), eliminating the need for key management and open inbound ports.

Terraform files, logs, and playbooks are stored in S3 under a hierarchical key structure of `{orgID}/{projectID}/{diagramID}/`, ensuring tenant isolation at the storage layer.

End-to-End Workflow

The complete workflow from authentication to deployment proceeds as follows:

- The user authenticates via Google OAuth 2.0 and receives a JWT session token.
- The user creates an organization and project through the dashboard.
- Using the React Flow canvas, the user drags and drops AWS resource blocks and connects them.
- The diagram is saved to PostgreSQL. When the user initiates deployment, the Terraform engine Lambda generates `main.tf` and `outputs.tf` (with auto-generated IAM policies from diagram edges) and uploads them to S3.
- The user clicks Apply. A Fargate task (`terraform-deployer`) runs `terraform init`, performs STS AssumeRole into the client's account, and executes `terraform apply`. Logs and state are uploaded to S3.
- If EC2 instances are present, an Ansible job is submitted to SQS. A Fargate `ansible-runner` uses SSM Run Command to execute the playbook directly on the target EC2.
- Deployment logs are written to S3 and status is recorded in the `diagram_deployment_logs` table. The frontend polls for updates via React Query with a 10-second refetch interval.
- If any step fails, the Fargate task writes the error to the `jobs` table (status set to `FAILED` with `error_message`), logs the failure to S3, and the frontend displays the error to the user on the next poll cycle. No automatic rollback is performed; the user can review logs and re-trigger or destroy.

See Figure 4.5 for more information.

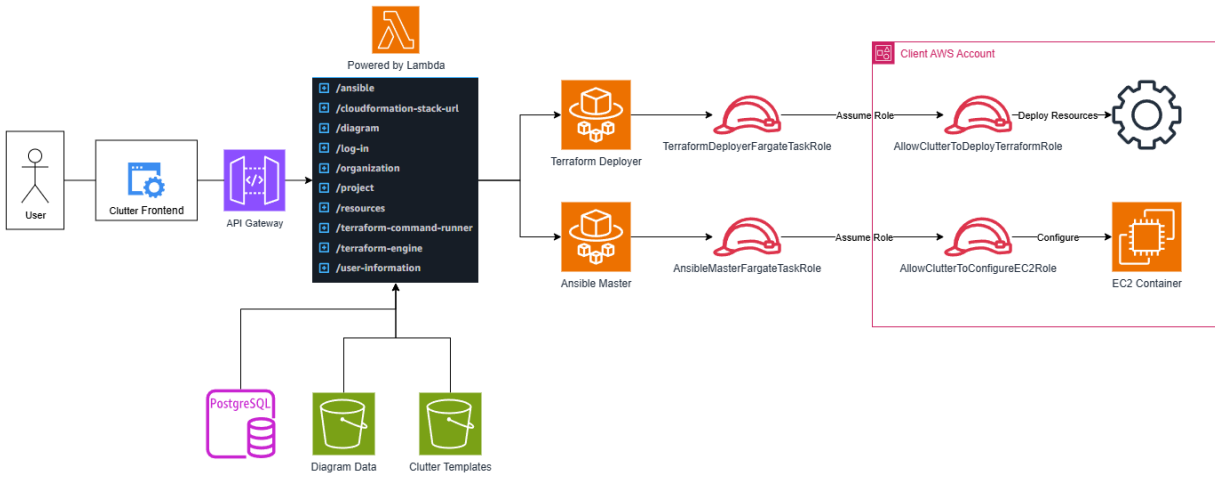


Figure 4.1: System Architecture.

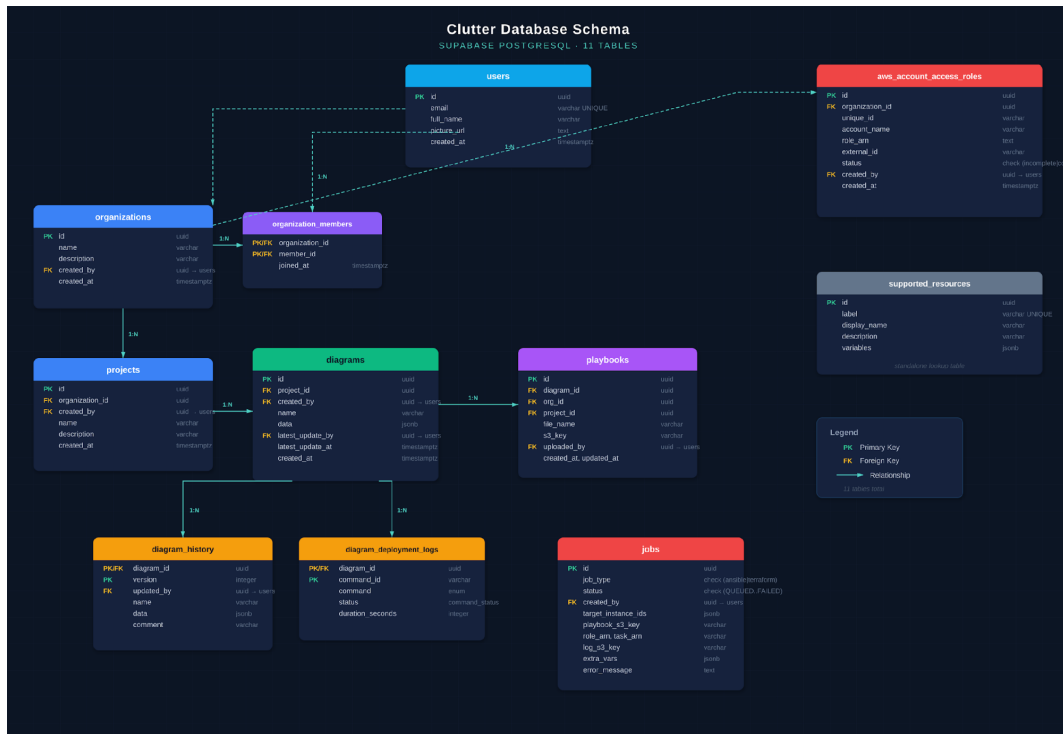


Figure 4.2: Clutter Database Entity-Relationshipship Layout.

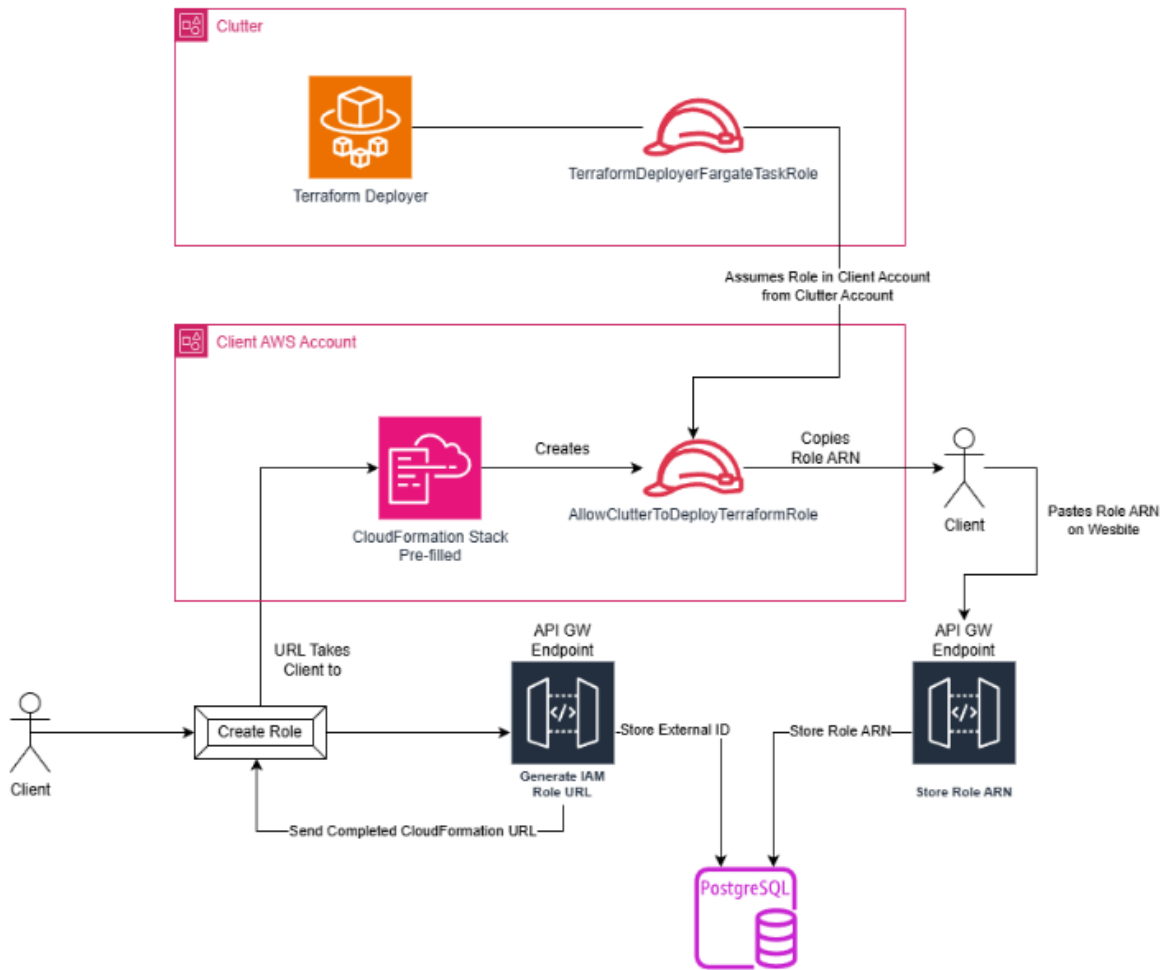


Figure 4.3: Terraform Deployer architecture with IAM cross-account role assumption.

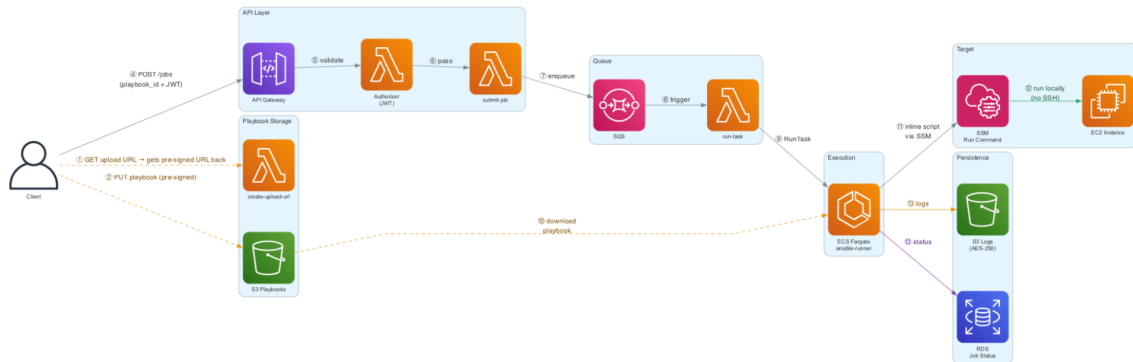


Figure 4.4: Ansible + Fargate + Playbooks pipeline architecture.

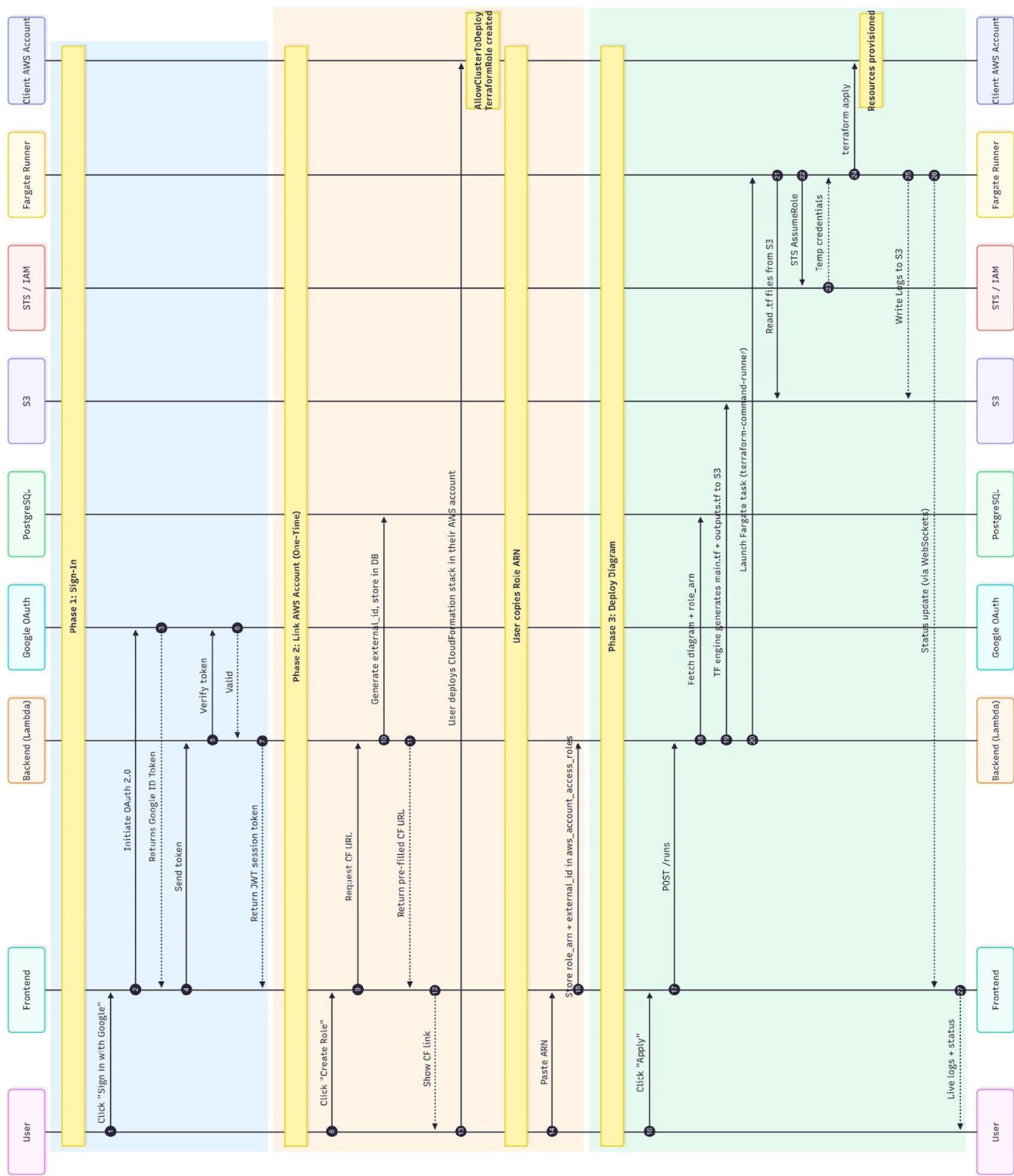


Figure 4.5: End-to-end authentication and deployment flow.

Project Scope and Final Product Functionality

The scope of this project was to achieve the core features which would accomplish our goal of making a cloud resource diagramming tool that abstracted away from programming. To ensure we met these goals in the timeframe of this capstone project, we had to streamline the scope. In doing so, we chose to stick to only one cloud provider (AWS), implement a small subset of cloud resources to our diagrams, and we had to cut a couple features we were originally planning to implement.

Scope

- Client sign on with Google OAuth 2.0
- Organization management (top level)
- Project management (under organization)
- Diagram management (under project)
- Client AWS account linking process
- Diagramming interface to drag and drop AWS services onto a canvas
 - Support the following resources
 - * API Gateway
 - * Lambda
 - Uploading of a .zip file containing all necessary source code and dependencies required for execution (stored in S3)
 - * EC2
 - Uploading of Ansible Playbook to run on instance (stored in S3)
 - * DynamoDB
 - * S3
 - Allow connections of relevant resources to one another
 - * Handle IAM permissions required to link resources
- Terraform Engine component which generates valid Terraform code given a diagram layout and stores it in S3

- Terraform Deployer component which intakes Terraform code inside of the S3 bucket and applies it to the client's AWS account
- Ansible Runner component which intakes an Ansible Playbook from S3 and executes it on a client EC2

Deviation of the Final Scope

We made two deviations from our original scope described in the Fall Design Roadmap.

For the first deviation, we decided not to include the feature that would allow users to extract the contents of an EC2 container and replicate it on a local machine. The intended functionality was to enable users to pull the full container environment from the cloud and run it locally with the same configuration and behavior.

The second deviation was not implementing collaborative features such as inviting users to an organization and allowing them to work in a shared diagramming environment.

Due to time constraints, we made the decision as a team to not proceed with implementing these features. Instead, we prioritized core functionality that was critical to delivering an MVP with a solid foundation. We formally communicated these scope changes to our sponsor and in our Winter Term Presentation, providing transparency to our academic advisor and teaching assistant regarding the rationale behind our decision.

Product Functionality

Client Sign On

All sign in functionality is managed using Google OAuth, as shown in Figure 5.1.

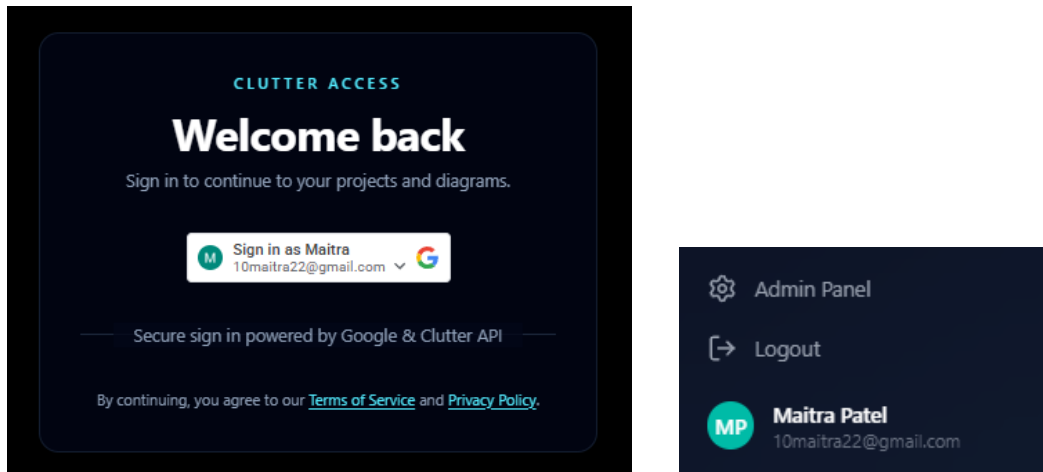


Figure 5.1: Google OAuth sign-in interface and user profile.

Organization Management

This is what a client sees when they sign in for the first time. They are prompted to create an organization to continue, Figure 5.2. Once the user creates or has an organization, they would see the organization main dashboard, Figure 5.3.

The image shows a 'Let's start by creating your organization' form with a dark blue background. The form has a title 'Let's start by creating your organization' with a plus icon. Below the title is a subtitle: 'This is where you'll create projects, diagrams, workspaces, and runs. You can invite teammates later.' The form contains two input fields: 'Organization name' with a placeholder 'e.g., Clutter Labs' and a note 'Must be 4-32 characters to match the backend validation.', and 'Description (optional)' with a placeholder 'e.g., Infrastructure diagrams for my team' and a note 'Optional. Up to 300 characters. Cannot start/end with spaces or contain double spaces.' There is a 'Create organization' button with a right arrow. To the right of the form are two informational boxes: 'Collaboration-ready' which says 'Invite members, assign roles (owner/editor/viewer), and collaborate on diagrams with presence + locks.', and 'Project isolation' which says 'Your org is the security boundary. Projects, canvases, nodes, and runs stay scoped to your org.' Below these is a 'What happens next?' section with a list: '1. Create your first project', '2. Build a diagram (Lambda / EC2 / DynamoDB / API GW)', and '3. Export Terraform or run Plan/Apply'. At the bottom right is a tip: 'Tip: If you're joining a team, you might already have an invite.'

Figure 5.2: Clutter Organization Create Form.

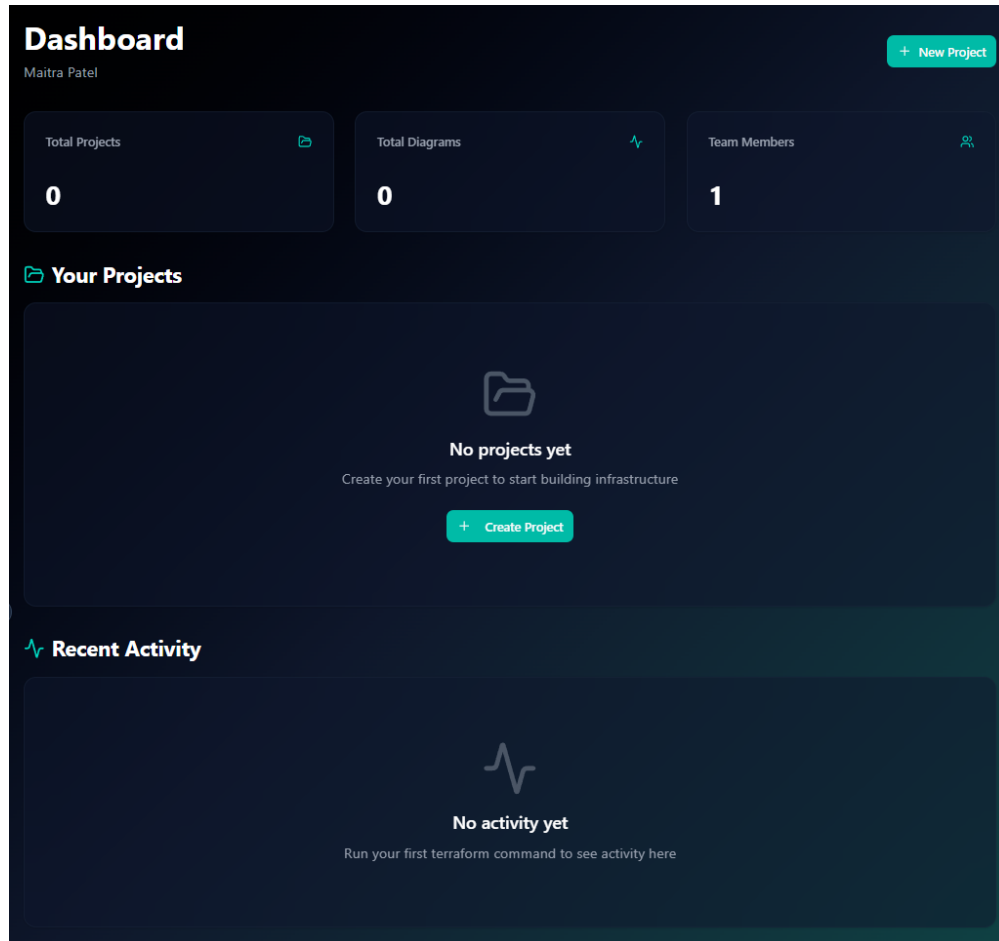


Figure 5.3: Organization Dashboard.

Project Management

Users are able to create a new project within the organization dashboard, Figure 5.4, then after they are able to choose the name of the project and create an optional description, Figure 5.5. Everything is managed through the project dashboard, see Figure 5.6.



Figure 5.4: Create Project from Dashboard.

Create New Project

Create a project to start building diagrams, variables, and workspaces.

Project name

Description (optional)

Create Project

Back to Dashboard

Figure 5.5: Create Project Form.

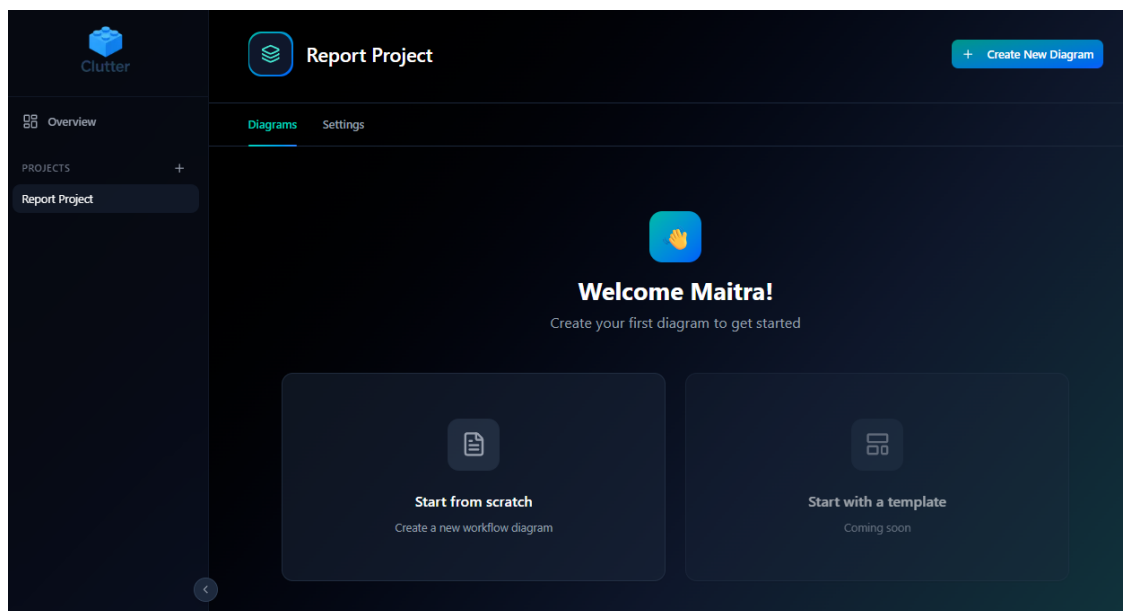


Figure 5.6: Project Dashboard.

Diagram Management

Within the project dashboard can create a diagram with the top bar, see Figure 5.7, or only if there are no diagrams through the start from starch button, see Figure 5.8.



Figure 5.7: Create Diagram.

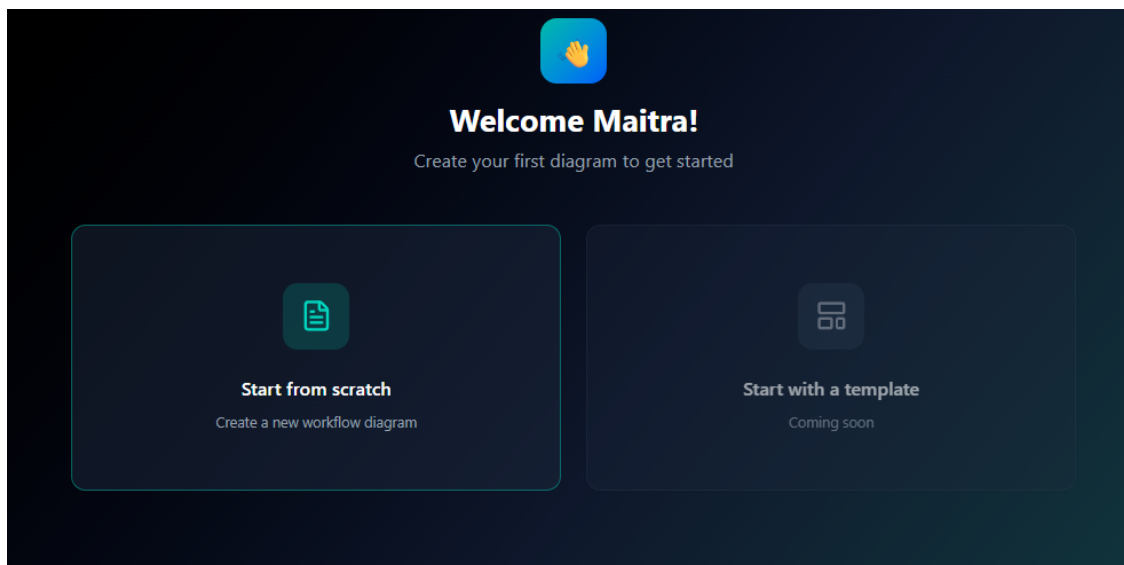


Figure 5.8: Create Diagram from Scratch.

AWS Account Linking Process

If a client tries to start a diagram before linking their AWS account, they are nudged to go do that first, as shown in Figure 5.9.

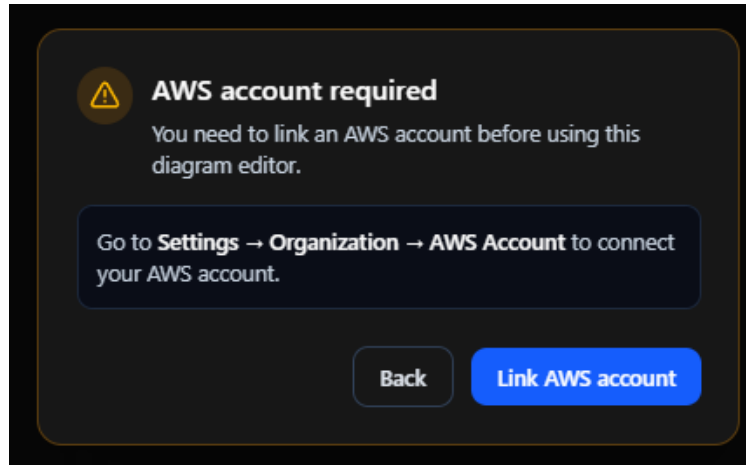


Figure 5.9: AWS Missing Account Warning within the Diagram Dashboard.

Within the organization settings, the user can access the AWS account tab where they are prompted to the linking AWS account setup, see Figure 5.11, there is an PDF walking through each step as well, as seen in Figure 5.12. Finally, the page changes to a status of their linked account once the steps are successfully completed, Figure 5.10.

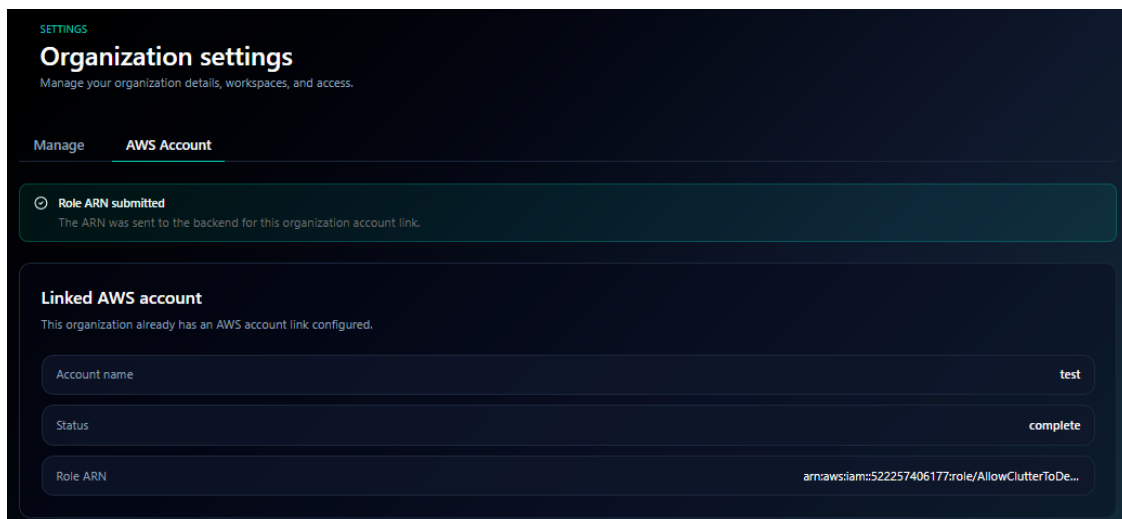


Figure 5.10: AWS Linked Account Page.

SETTINGS

Organization settings

Manage your organization details, workspaces, and access.

Manage

AWS Account

Connect an AWS Account

Clutter deploys infrastructure into your AWS account by assuming a secure IAM role. Create the role in AWS, paste the Role ARN here, then verify and save the connection.

No long-lived AWS keys

Frontend only stores metadata like alias, role ARN, and region.

Need a walkthrough? Open the step-by-step AWS account linking guide.

View setup guide

1. Open AWS setup

Enter the account name first. Clutter will generate the CloudFormation launch URL and open AWS for you.

AWS account name

e.g. DemoAccount

This becomes the 'accountName' query parameter for the setup-link endpoint.

Deployment region

us-west-2

If omitted, the API defaults to 'us-west-2'. Keeping it explicit is clearer for the user.

Open AWS setup in CloudFormation

2. Paste role details and submit

After you finish in AWS, paste the Role ARN here and submit it. Format validation is optional.

AWS account name

This comes from the setup-link step and is sent to the backend before the ARN step is unlocked.

Role ARN

arn:aws:iam::123456789012:role/ClutterDeployRole

Submitting this sends 'role_arn' to the backend account endpoint.

Default region

us-west-2

Submit ARN

Optional: Validate ARN format

Connection preview

This reflects the current setup state for this account connection.

Status	Waiting for input	AWS account name	Not set	Region	us-west-2
Pending link ID	Not created yet	Launch URL	Not generated yet		

Generate the setup link in AWS first, then paste the Role ARN here to complete the connection.

Figure 5.11: AWS Linking Account Page.

Need a walkthrough? Open the step-by-step AWS account linking guide.

View setup guide

Figure 5.12: PDF Walking Through Link.

Diagramming Interface

The diagramming interface, shown in Figure 5.13, provides a drag-and-drop canvas powered by React Flow. A collapsible resource palette on the left organizes available AWS services into three categories: Compute (Lambda, EC2 Container), Storage (DynamoDB, S3), and Network (API Gateway).

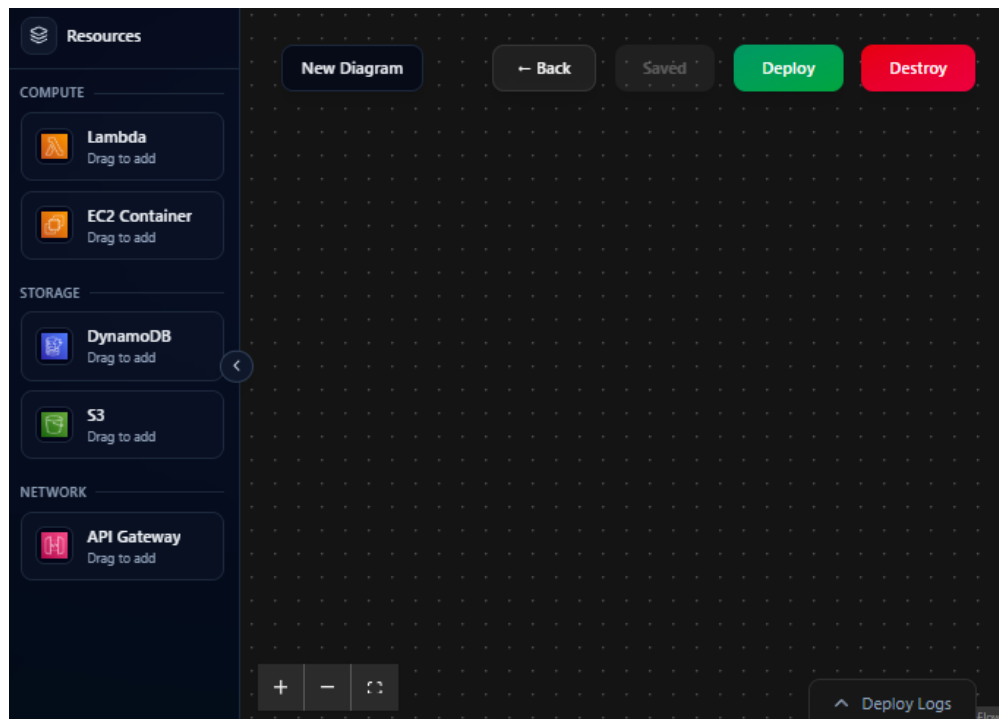


Figure 5.13: Diagram Canvas Page.

The toolbar at the top of the canvas, shown in Figure 5.14, allows the user to navigate back to the dashboard, save the current state, trigger a deployment, or destroy previously deployed infrastructure. The Destroy button is only available once the diagram has been deployed at least once.

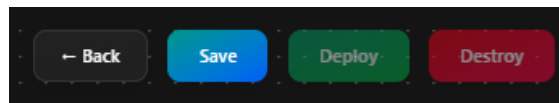


Figure 5.14: Diagram Top Nav Settings.

Each resource block displays contextual validation hints when required configuration fields are missing, as illustrated in Figure 5.15. Clicking a resource opens a configuration panel specific to that resource type. Figure 5.16 shows the API Gateway panel, which exposes fields for resource

name, description, stage name, CORS settings, and supported HTTP methods. Every resource type has its own set of configurable fields derived from the `supported_resources` table.

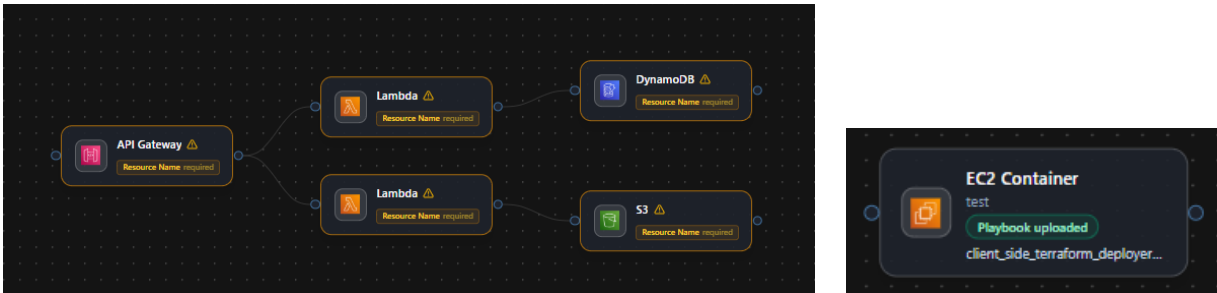


Figure 5.15: Diagram Resources Blocks.

The sidebar shows the configuration for the 'AWS API Gateway' resource. It includes the following fields and controls:

- RESOURCE NAME ***: A text input field containing 'test'. Below it, a hint reads 'Terraform resource ID and API name'.
- DESCRIPTION**: A text input field containing 'API description'. Below it, a hint reads 'API description'.
- STAGE NAME**: A text input field containing 'v1'. Below it, a hint reads 'Deployment stage name'.
- ENABLE CORS**: A toggle switch set to 'True'. Below it, a hint reads 'Generate OPTIONS method and CORS headers'.
- HTTP METHODS**: A group of checkboxes for 'GET', 'POST', 'PUT', 'PATCH', and 'DELETE'. The 'GET' checkbox is checked.

Figure 5.16: Diagram Resource COnfig Sidebar.

For EC2 nodes, an Ansible Playbook can be uploaded and associated with the instance. When a playbook has been uploaded, this is reflected directly on the resource block on the canvas, as shown in Figure 5.15. When the user clicks Deploy, the interface transitions to a live log stream fed directly from the AWS Fargate task, as shown in Figure 5.17, allowing the user to monitor the deployment in real time. Upon completion, all log files are persisted to S3 and remain accessible to the client through the Log Files tab, shown in Figure 5.18, which displays each command alongside its status, timestamp, and duration.

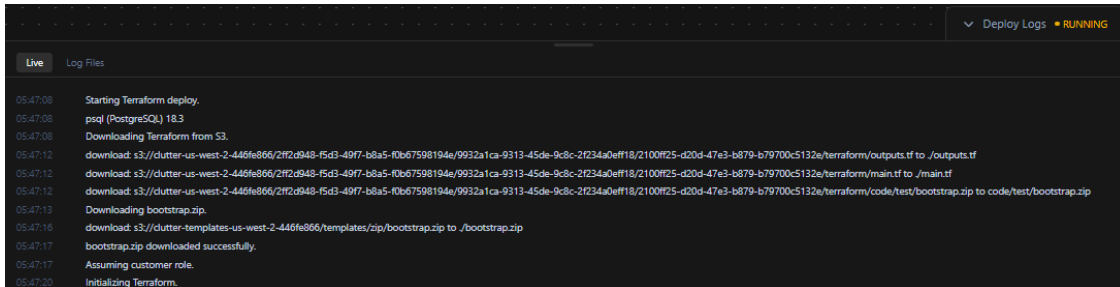


Figure 5.17: Diagram Top Nav Settings.

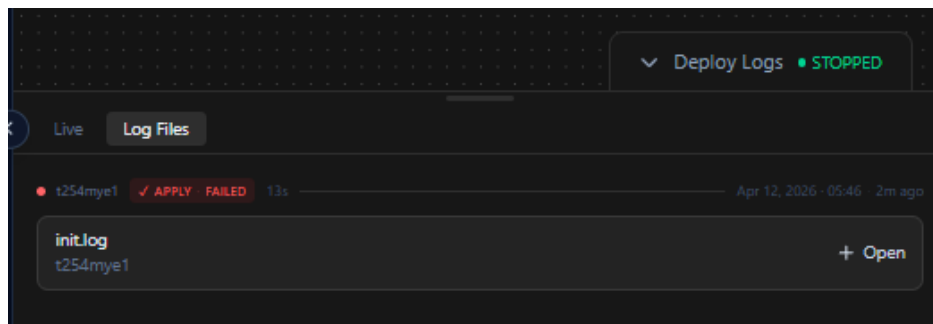


Figure 5.18: Diagram Top Nav Settings.

Terraform Engine

When the diagram is saved, the frontend sends a payload to the backend with the new diagram structure. This diagram structure is saved in the diagram's PostgreSQL table. When the diagram table updates, we automatically generate and save a Terraform code file in S3, as illustrated in the JSON in Listing 5.1.

The "main.tf" and "outputs.tf" file gets added or updated in S3 every time a diagram "Save" is pressed, see example in Figure 5.19

Listing 5.1: Example request body.

```
{
  "edges": [],
  "nodes": [
    {
      "id": "de8f17f0-40b1-41d5-bc09-9510444e660a",
      "data": {
        "img": "/aws/lambda.png",
        "label": "Lambda",
        "variables": {
          "handler": "main",
          "runtime": "provided.al2",
          "timeout": 3,
          "memory_size": 128,
          "architecture": "arm64",
          "resource_name": "test2"
        }
      },
      "type": "awsService",
      "measured": {
        "width": 137,
        "height": 66
      },
      "position": {
        "x": 800,
        "y": 620
      }
    }
  ]
}
```

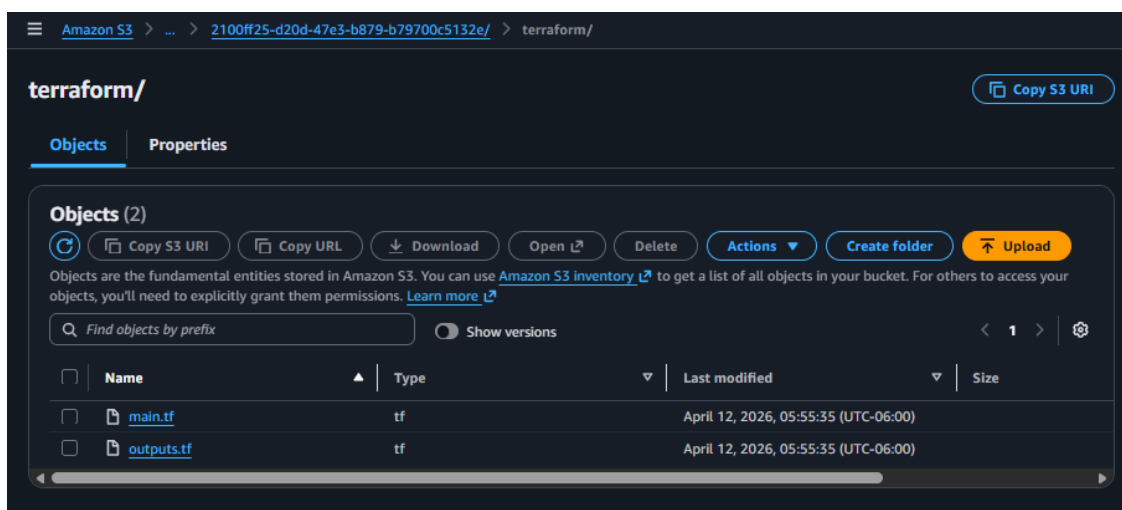


Figure 5.19: Auto-generated Terraform Files in S3 Bucket.

Terraform Deployer

When the “Deploy” or “Destroy” buttons are pressed on the frontend, they call the Terraform Deployer API which initiates the request by invoking an AWS Fargate task. If it is deploying, the Fargate task will download the Terraform code found in S3 and apply it to the client’s AWS account.

To illustrate this, a deploy process has been initiated on a single Lambda function named `test242`. Figure 5.20 shows the resulting log files after a successful application, with the status recorded as `APPLY SUCCESS`. Figure 5.21 confirms the Lambda function was successfully provisioned in the simulated client AWS account.

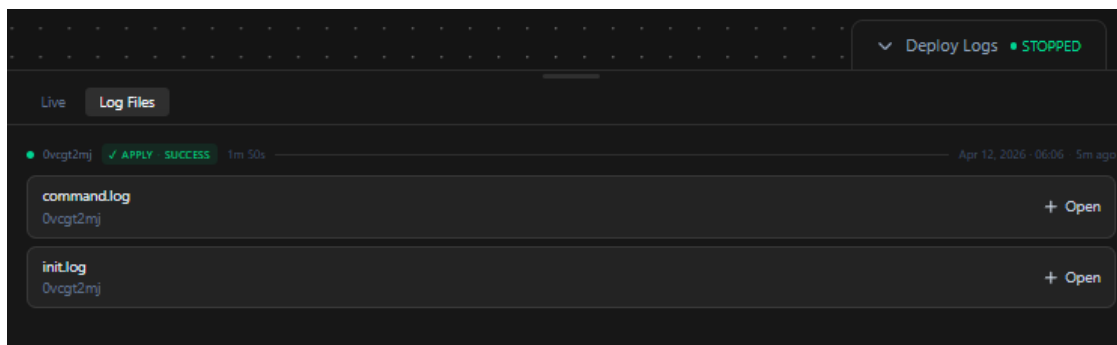


Figure 5.20: Example Lambda Apply Success Logs.



Figure 5.21: Example Lambda Deployment in AWS.

If it is destroying infrastructure, the Fargate task will download all necessary Terraform code and state files from S3 to destroy the resources from the client’s AWS account properly. Figure 5.22 shows what the `terraform` folder looks like in S3 after a successful deployment, containing `main.tf`, `outputs.tf`, `terraform.tfstate`, and a `logs/` folder.

☐	Name	Type
☐	.terraform.lock.hcl	hcl
☐	logs/	Folder
☐	main.tf	tf
☐	outputs.tf	tf
☐	terraform.tfstate	tfstate

Figure 5.22: Example Lambda S3 Objects.

To demonstrate destruction, the same `test242` Lambda deployed in the previous example was destroyed. Figure 5.23 shows the resulting log files with the status recorded as `DESTROY SUCCESS`. Figure 5.24 confirms that `test242` no longer exists in the simulated client account, with the AWS console returning no matches when filtering by that function name.

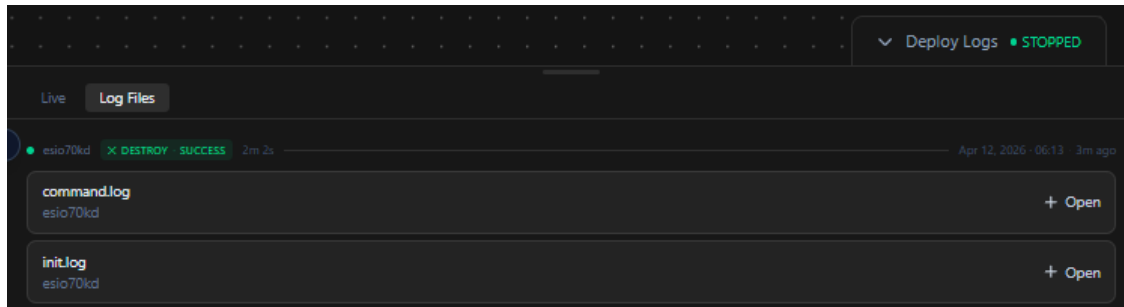


Figure 5.23: Example Lambda Destroy Logs.

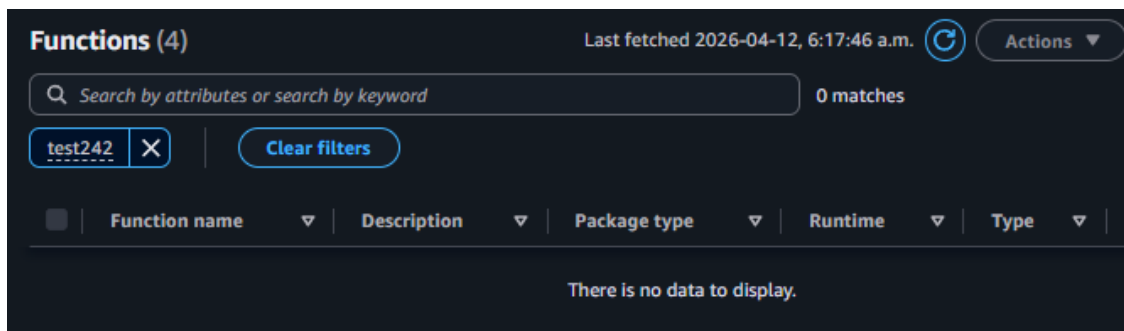


Figure 5.24: Example Lambda AWS Functions Console Empty.

Ansible Runner

Following the resources in Figure 5.15, the configuration panel illustrated in Figure 5.16 and Figure 5.25 allows the user to upload an Ansible Playbook for EC2 resources. Once a playbook is uploaded and an instance ID is provided, the client can run the playbook on that instance, seeing in Figure 5.25.

The image displays two side-by-side screenshots of the 'ANSIBLE PLAYBOOK' configuration panel. Both panels have a dark theme with light text.

Left Panel (Initial State):

- Bound EC2 container:** EC2 Container
- Action:** Upload playbook for this EC2 container (indicated by an upload icon in a dashed box)
- Uploaded file:** No playbook uploaded yet
- TARGET EC2 INSTANCE ID:** i-0a05920cb52c4555d
- Playbook ID:** Not available until upload succeeds
- Buttons:** Upload a playbook to run (disabled), Run playbook on this EC2 container (disabled)
- Status:** Uploaded "client_side_terraform_deployer_role.yaml" for this EC2 container.

Right Panel (After Upload):

- Bound EC2 container:** EC2 Container
- Action:** Replace playbook for this EC2 container (indicated by an upload icon in a dashed box)
- Uploaded file:** client_side_terraform_deployer_role.yaml
- TARGET EC2 INSTANCE ID:** i-0abcd1234efgh5678
- Playbook ID:** f91e28c7-0059-45b7-921d-20d80bfa7e73
- Buttons:** Upload a playbook to run (disabled), Run playbook on this EC2 container (active)
- Status:** Uploaded "client_side_terraform_deployer_role.yaml" for this EC2 container.

Figure 5.25: Ansible Runner Config Panel

Technical Specifications

Table 6.1: System Requirements, Measurement Criteria, and Evaluation Methods

Specification	Requirement	Measurement Criteria	Evaluation Method
Canvas Editor	Create, move, and delete AWS service nodes and edges in a diagramming interface. <i>Deviation: Saving is done manually and no undo/redo feature. This was removed from the scope as it was deemed not as important as core features.</i>	All actions are clear and intuitive in the UI. Saving calls its correct API and stores data in the PostgreSQL table.	Functionality is tested using manual and exploratory testing on the frontend UI. Same is done for ensuring saving works correctly.
AWS Account Linking	Clients are able to create an organization and go through the account link setup process. Detailed steps are provided on the frontend in case they are confused.	The process should be easy and intuitive for a client to complete on their own.	Ask an external person to go through the steps to link an AWS account and see if they are able to do so without being blocked.
Core APIs	All necessary frontend actions bound to an API endpoint. API is made using API Gateway with each endpoint linked to a Lambda function.	Endpoints fulfill the expectations set by the frontend, whether that be to create, retrieve, update, delete records, or invoke a task such as the Terraform Deployer or Ansible Runner.	Functionality testing through manual and exploratory testing. The API testing suite is Postman with an array of sample requests and saved responses as documentation.

Specification	Requirement	Measurement Criteria	Evaluation Method
PostgreSQL Database	Use a PostgreSQL database hosted on Supabase to store record data such as users, AWS account data, organizations, projects, diagrams, logs, and Ansible playbook data. <i>Deviation: Originally planned on using DynamoDB but due to growing complexity, little domain knowledge, and heavy relational data, switched to PostgreSQL.</i>	Tables are adequately normalized reducing redundancy. All data is created, read, updated, and deleted through the API to prevent stale or unused records.	Complete tests on the respective API on redundancy by running sample queries to check for duplicates. Review the schema to ensure normalization is done at least up to Second Normal Form (2NF), with a goal for a majority to be at Third Normal Form (3NF).
Storage of Files in S3	All file storage is to be done inside of S3. Generic files are stored in a public templates S3. Client specific artifacts such as generated Terraform code, uploaded .zip files for Lambda, and Ansible Playbooks are stored in a private S3 bucket.	The templates S3 should be public. Client specific artifacts S3 should be private, with access mediated through the APIs. Pre-signed URLs are used for client uploads.	Access to private S3 is managed via Terraform IaC and verified through manual testing. All files stored in S3 have CRUD to ensure no stale or unused files.

Specification	Requirement	Measurement Criteria	Crite-	Evaluation Method
Terraform Engine	A webhook on Supabase to check for any live updates to the data column on the diagram table. This triggers a Lambda to generate and store Terraform code in S3 based on the diagram structure that was saved.	Generated Terraform code is valid and is a 1:1 representation of the diagram and provided fields.		Functionality is tested through manual and exploratory testing. We ensure that whenever a diagram is saved, the S3 gets a newly generated <code>main.tf</code> file. The code file is then deployed to verify correct infrastructure.
Terraform Deployer	When a Deploy or Destroy request is made on the frontend, an API is called to invoke the deployment Fargate task. This task downloads the current contents of the S3 <code>terraform</code> folder and runs either the <code>apply</code> or <code>destroy</code> command targeting the client AWS account.	Whether a deploy request creates the correct infrastructure on the client account. Whether a destroy request destroys all infrastructure that was configured.		Functionality is tested through manual and exploratory testing using a simulated client account. We configure the AWS account link to this account and run deploy and destroy commands to verify infrastructure is managed correctly.

Specification	Requirement	Measurement Criteria	Evaluation Method
Ansible Runner	If a client wants to configure an EC2 container, they can upload a valid Ansible Playbook to the EC2 in the UI. If the EC2 is already deployed, the client pastes the instance ID and runs the playbook, spinning up a Fargate task that downloads and executes the playbook.	Fargate task downloads the playbook correctly, serves as an Ansible master, targets the EC2 instance, and runs all defined configurations in the playbook.	Functionality is tested through manual and exploratory testing. We run a sample Ansible playbook which configures a web server on the target EC2 and check for the existence and accessibility of this web server.
Run Logging	Live status of deployment and Ansible runs. These should be clearly visible in the diagram UI and allow clients to see the history of deploy and destroy with detailed log files.	Logs are available within 2000ms of emission from CloudWatch logs. Logs are displayed on the frontend automatically and refreshed every 2000ms.	Functionality is tested through manual and exploratory testing. We run deployments and destructions and note the logs for correctness. The source of truth is CloudWatch logs.
Error Handling	Consistent error models which include accurate error codes and messages which surface on the UI to provide clear failure responses.	All endpoints must return structured errors consistent with the agreed model, enabling the frontend to display actionable messages for every error case.	Functionality is tested through exploratory testing. We run APIs with known error cases and observe the output and how the frontend error display system handles it.

Specification	Requirement	Measurement Criteria	Evaluation Method
Documentation	Detailed README files for each crucial code section in GitHub. A section to show how to deploy the application. A walkthrough video. A thorough log of known bugs and edge cases. <i>Deviation: Increased goal from 30 minutes to 2 hours as the system is more complex than anticipated.</i>	The sponsor and/or a new team can pick up the work easily, within 2 hours of thorough reading.	Hand the completed documentation to an external person and afterwards ask: How easy is the documentation to follow? Does it make sense with little pre-existing domain knowledge?
Budget Constraints	The application must operate with the smallest available budget as possible, relying on free tiers when available. If not, reducing hardware configurations to only what is needed.	Compliance with less than \$50 budget. Since architecture is entirely serverless, cost scales with users.	Regular budget assessments through AWS Billing and Cost Management.

Local Dev Environment

This is not included in the final technical specifications because we did not end up configuring formal local environments for development. We misunderstood what a development environment would look like since we were using a serverless architecture. Most of what was configured was on the cloud and not locally. What was local was the Terraform IaC which we would deploy and our Go source code. Both of these required minimal environment setup.

Access Control (IAM)

While we did use least-privilege roles for our services to ensure they had just the right permissions they needed, we did not use IAM Access Analyzer to scan from AWS. We felt that this was an unnecessary step as we were already managing all of our infrastructure with Terraform and had full visibility on which resources had which permissions. We did not feel that it needed an extra layer of validation as it would tell us what we already know with our IaC.

Build and CI Time

We did not end up creating a CI pipeline for this project as it would have proved to be extra work which we would not have bandwidth for. However, going through the steps manually, we still meet the goal of completing the full application frontend and backend end-to-end completeness in under 10 minutes.

Measures of Success and Validation Test Results

We evaluated Clutter as an application against the product scope and technical specifications outlined earlier in this document. The primary metrics we used to determine success were whether we successfully delivered a web-based, drag-and-drop interface that allows users to describe a target AWS infrastructure, generate the corresponding Terraform and Ansible artifacts, and deploy that design to a client AWS environment.

In summary, if a user could create a multi-component AWS setup without manually writing infrastructure-as-code, we considered our minimum viable product (MVP) to be achieved. After discussions with our sponsor, Devin Atkin, we also identified and refined key success metrics based on their expectations and feedback.

Functional Validation Against Sponsor Expectations

These were the core features our sponsor, Devin Atkin, required. Specifically, a user should be able to log in, create or open a project, build a diagram using pre-planned AWS resource blocks, save that diagram, and trigger infrastructure generation and deployment.

This functionality was primarily tested through frontend-driven end-to-end workflows, which were completed during the Fall semester. Success was determined by verifying that diagrams persisted after a page refresh, that status and log outputs became visible to the user, and that the generated Terraform successfully deployed the infrastructure, with the resulting resources appearing correctly in AWS. Based on these validation steps, we were able to confirm that these core requirements were successfully met.

- Creating a new organization and having it successfully persist in the database
- Creating a new project and having it appear in the dashboard
- Switching between projects and loading the associated diagrams
- Creating a new diagram canvas
- Saving a diagram's nodes and edges
- Refreshing the page while preserving the diagram state
- Editing an existing diagram and updating the backend
- Deleting a project or diagram and removing it from both the UI and backend

Technical Validation

API and Backend correctness

The next part was the validation of our backend. This was tested using a comprehensive set of Postman API tests, covering both CRUD operations and workflow endpoints. We sent both valid and invalid requests to verify correct behavior across happy paths and edge cases, ensuring that all scenarios were handled appropriately. These tests were organized by backend functionality, allowing the team to systematically manage and validate each set of test cases.

- All CRUD endpoints (Projects, Canvas, Nodes, Edges) returned correct responses
- Valid API requests returned expected JSON structures
- Invalid API requests returned structured error messages
- Authentication-protected endpoints rejected unauthorized requests
- JWT-based login returned valid tokens and user data
- Backend correctly handled missing or malformed payloads
- Concurrent updates to the same resource did not corrupt data
- API latency remained low under repeated requests

Figure 7.1 and Figure 7.3 are an example of how we tested whether an unknown project ID would be handled correctly by the system. In this case, the API was expected to reject the request and ensure that no unintended resources were created in the backend.

These were tested and handled before connecting them to the frontend to ensure that we followed the correct processes and maintained rigorous testing practices.

Terraform Generation and Deployment

We also had to test out our terraform generation with the following key metrics to be validated

- Valid diagrams generated Terraform code successfully
- Generated Terraform reflected correct resource relationships
- Identical diagrams produced consistent Terraform output

- Changes to diagram updated Terraform output accordingly
- Terraform files were correctly stored/retrieved to the S3 in the backend

Figure 7.2 is an excerpt from our application demonstrating a successful connection to the backend and a successful deployment of the Ansible script running on AWS.

Usability Testing for Non-Technical Users

We tested our final application with non-technical users who were colleagues and not stakeholders in the project. Through this testing, we verified that non-technical users were able to create a simple diagram without guidance, understood the drag-and-drop interactions, and could correctly identify AWS components (such as distinguishing between Lambda functions, S3 buckets, and other resources). Importantly, users were able to use the system without any prior knowledge of Terraform.

This testing also included exploratory testing, where users were free to navigate and interact with the system in order to uncover edge cases or missed scenarios not captured by our structured test cases. Any issues or gaps identified during this process were documented separately for further analysis and improvement.

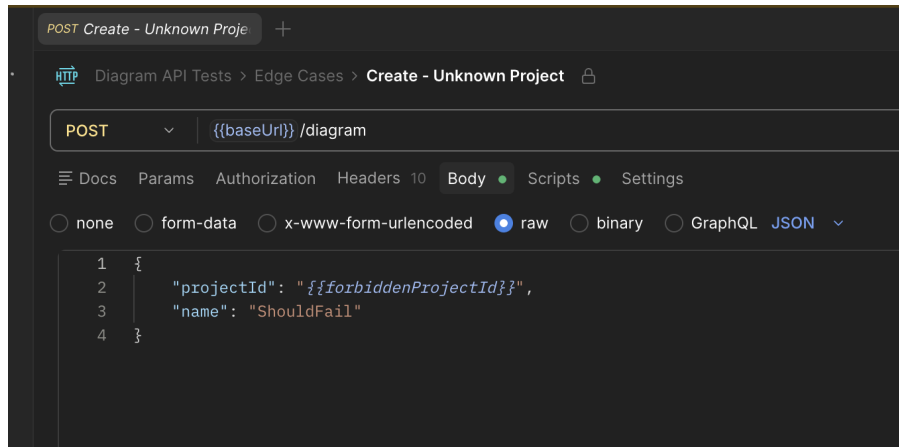


Figure 7.1: Postman Test Example.

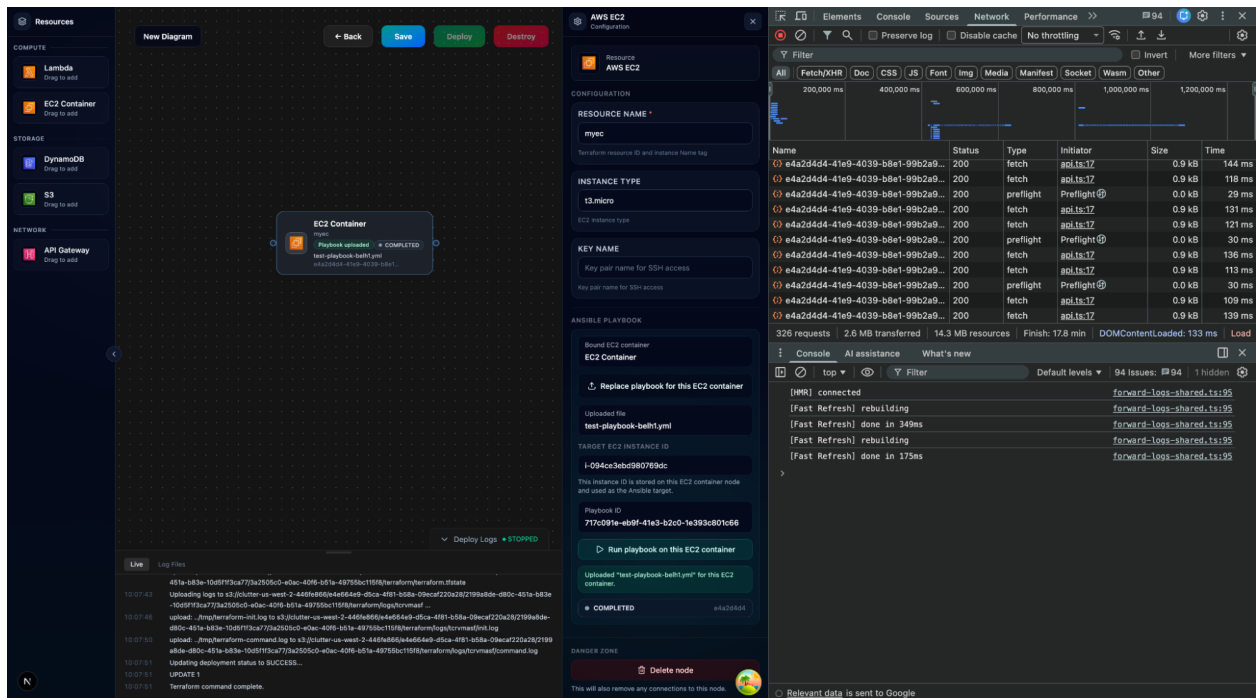


Figure 7.2: Frontend Network Test.

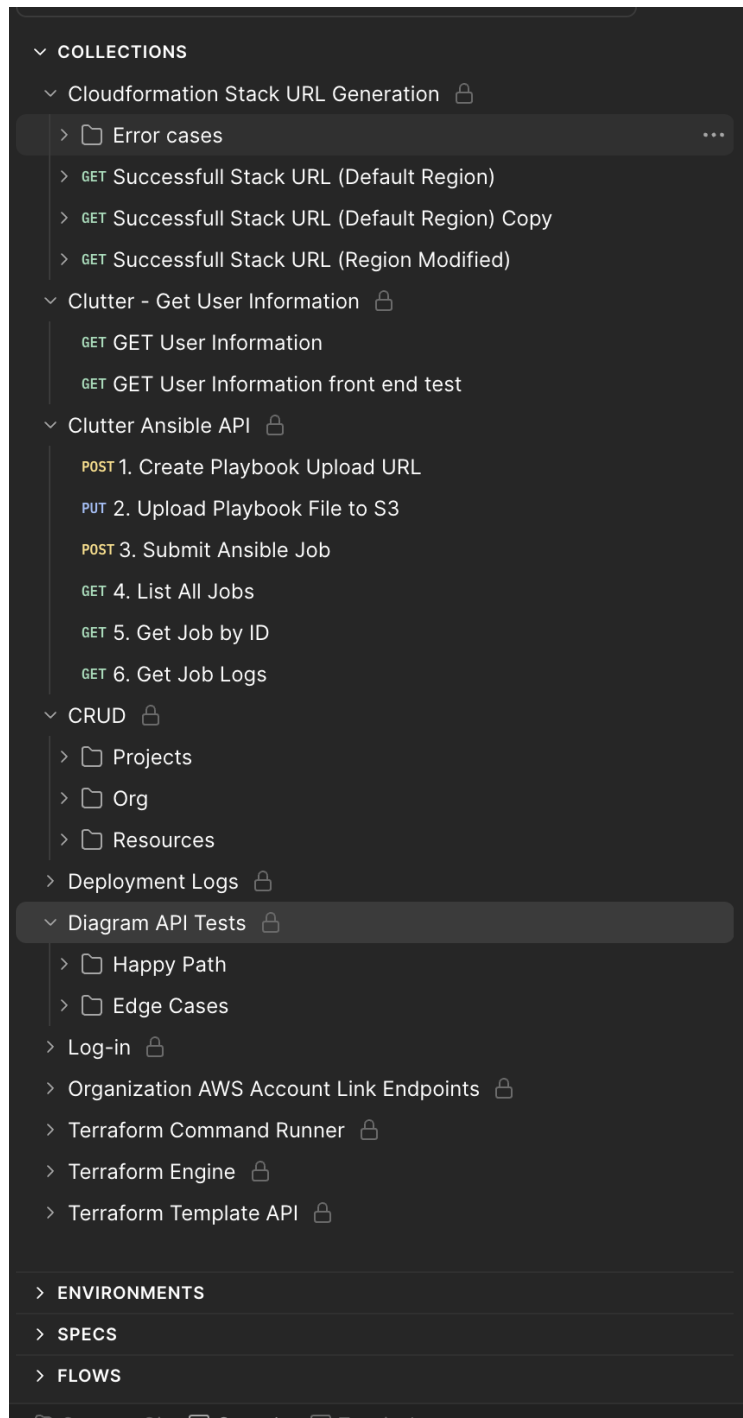


Figure 7.3: Postman API Collection.

Tools, Materials, Supplies, and Cost

Clutter was a software-only project, so no physical materials, supplies, or hardware were needed. This matches our Fall term report, where we stated that the project would not require physical materials or hardware and that the main possible costs would come from cloud services, while most tools and libraries would be free or open-source. The finalized list of tools and services used in the project includes:

- VS Code
- Next.js
- React
- TypeScript
- Tailwind CSS
- React Flow
- Go
- Docker
- GitHub Actions
- Terraform
- Ansible
- Postman
- AWS Lambda
- API Gateway
- S3
- SQS
- ECR
- CloudWatch Logs
- AWS Fargate
- Supabase PostgreSQL

The frontend was hosted on a sponsor-provided VM using Docker Compose and the domain was also provided by the sponsor.

The actual cost was still very low, but not exactly zero. The capstone AWS account ended at about \$0.20 and there was also an additional small AWS charge from Ansible testing of \$0.72. AWS testing was completed using test accounts with free credits, Supabase was used on its free tier and no paid tiers of software or services were used. This means the actual final cost was lower than the possible cloud-service cost anticipated in the Fall term report. In the Fall report, we expected possible AWS-related costs from services such as Lambda, DynamoDB, EC2 and S3, although we also noted that these would likely be reduced through AWS free tier usage or credits.

The main deviation from the Fall term report was not the budget, but the architecture. In the Fall term, we planned to use DynamoDB as the main database. In the final product, we switched to

PostgreSQL hosted on Supabase because the project data became more relational and PostgreSQL was a better fit. We also ended up using additional AWS services such as SQS, ECR and CloudWatch Logs as the final system became more complete. This deviation is justified because it improved the final design and still kept the project at no direct cost to the team.

Cost will increase if the number of users and deployments increases, because several of the AWS services we use are billed based on usage. For example, AWS Lambda charges by request volume, with request charges listed at \$0.20 per million requests on the official AWS pricing page. API Gateway pricing depends on API calls and data transfer and AWS shows example pricing of \$1.00 per million HTTP API requests in one tier and \$3.50 per million REST API requests in one example region/tier. S3 storage is also usage-based and AWS lists standard S3 storage example pricing in the range of roughly \$0.023 per GB-month for common tiers. Fargate is one of the more important scaling costs because it charges for vCPU-seconds, memory and storage consumed by each task.

Because of this, Clutter's current cost is very low only because usage is low and testing stayed within free credits and sponsor-supported infrastructure. In a real production setting, the cost would scale mainly with the number of API calls, deployment runs, stored artifacts and logs in S3 and the number and runtime of Fargate deployment or Ansible jobs. A reasonable ballpark conclusion is that the platform is inexpensive at small scale, but costs would rise gradually as more users create projects, save diagrams, trigger deployments and stream logs

Reflections

Ethics and Equity in our Capstone Journey

Building Clutter raised some important ethical questions that the team had to think carefully about. Because Clutter takes a user's AWS infrastructure diagram and turns it into working Terraform code, it handles information that could be sensitive. For example, a diagram showing how a company's internal systems are connected could reveal private business details. From the start, the team felt responsible for making sure that each user's data was kept private and separate from other users' data. This is why we chose to use Supabase's Row-Level Security feature and store each user's generated Terraform files in their own separate folder in S3, rather than mixing everyone's files together.

Another ethical concern was around the code Clutter generates. Because that code can be deployed directly to a real AWS account, any mistakes in it could cost a user real money or create security problems. The team addressed this by checking generated code against official Terraform templates and by adding clear warnings in the app telling users to review the diagrams before applying it. We felt it was important to be honest about the tool's limitations rather than make it seem more reliable than it is.

On the topic of fairness, one of the main reasons we built Clutter in the first place was to make cloud infrastructure tools more accessible. Currently, setting up cloud infrastructure with Terraform requires a lot of specialized knowledge that most developers do not have. This means smaller teams and students often cannot use these tools effectively. Clutter was designed to change that by letting anyone create infrastructure setups visually, without needing to know Terraform syntax. This goal shaped many of our design decisions, such as making the interface drag-and-drop, writing error messages in plain English, and making it easy to get started without any prior experience.

Within the team itself, we tried to make sure work was divided fairly based on each person's strengths. We checked in regularly to make sure no one was overwhelmed and that everyone felt comfortable speaking up if something was not working.

Impact of Engineering on Society and Environment

Clutter has the potential to make a positive difference in how people build and manage cloud infrastructure. Right now, there is a shortage of engineers who know how to set up cloud systems using tools like Terraform. This makes it hard for smaller companies, student groups, and non-

profits to use these tools, since they often cannot afford to hire specialists. Clutter helps solve this problem by letting developers create cloud setups visually, which lowers the skill barrier and makes these tools available to more people.

However, there is also a potential downside worth acknowledging. Because Clutter makes it easier and faster to add cloud services, users might end up creating more resources than they actually need. Cloud computing uses a significant amount of electricity and water to run data centres, so wasting cloud resources has a real environmental cost. The team tried to address this in two ways. First, the default settings in the Terraform code that Clutter generates follow AWS best practices and use appropriately sized resources rather than the largest available options. Second, the app clearly shows users how many resources their diagram includes, so they can think twice before generating and deploying everything.

Team's Approach to Project Management

The team kept project management simple but consistent. Our main coordination tool was a weekly standup meeting where each member shared what they were working on, what was blocking them, and what they planned to do next. This was important because Clutter had many moving parts, a React frontend, a Go backend running on AWS Lambda, a Terraform code generator, and cloud infrastructure and it was easy for work to become disconnected if we were not communicating regularly. The weekly meetings helped us catch problems early and adjust plans when needed.

We used GitHub to manage our code and track progress. Pull requests were not just for reviewing code, they also helped us explain what each change was doing, which made it easier for teammates to understand work that was outside their main area. We set up clear branching rules early in the project and followed them throughout, which prevented a lot of potential merge conflicts.

For risk management, we thought about three main types of risk. Technical risk covered things like new tools or integrations we had never used before. Schedule risk covered the fact that we all had other courses and commitments competing for our time. Scope risk covered the temptation to keep adding support for more and more AWS services. To manage technical risk, we made sure to build and test the core connections between services early, before building features that depended on them. To manage scope risk, we agreed on a fixed list of supported AWS resources and stuck to it.

Another significant challenge was a database technology switch mid-project. The team originally built Clutter using a NoSQL database, but we ran into ongoing difficulties working with non-relational data structures as the project grew more complex. After weighing the options, we made the decision to switch to PostgreSQL. This meant refactoring a significant portion of the backend to work with the new database structure. It was a costly decision in terms of time, and it

added pressure to an already busy schedule. However, it turned out to be the right call. The team had much stronger existing knowledge of SQL and relational databases, and once the switch was made, development moved noticeably faster. The lesson here is that sticking with a technology just to avoid a painful refactor is not always the best choice. Sometimes taking a step back to fix a foundational decision saves more time in the long run than pushing forward with something that is slowing the team down.

The biggest challenge we ran into was time management, which directly connected to the schedule risk we had identified at the start. As full-time students with other courses, assignments, and exams competing for our attention, there were periods where progress on Clutter slowed down significantly. This led to stretches of crunch time where the team had to put in a lot of hours in a short window to catch back up before deadlines. Looking back, the weekly standup helped us stay aligned on what everyone was doing, but it did not do enough to enforce consistent progress week to week. The lesson we took from this is that identifying a risk is not the same as managing it. We recognized schedule risk early but did not put a concrete plan in place to deal with it, such as setting hard weekly targets or flagging early when someone was falling behind due to other coursework. In a professional setting, this kind of risk would need an actual mitigation plan attached to it, not just an acknowledgment. That is something each team member will carry into future projects.

Team's Approach to Economics and Cost Control

Keeping costs low was a priority for the team, both because we were working on student budgets and because it did not make sense to spend a lot of money on a tool that is itself meant to help people manage their cloud costs.

Our main strategy was to use the AWS Free Tier as much as possible. Services like Lambda, S3, and API Gateway all have free usage limits that are more than enough for our development and testing needs. We also used Supabase's free plan for our database, which saved us from paying for a separate database server.

The one area where costs went higher than expected was AWS Fargate. Fargate runs containerized workloads and charges based on how much CPU and memory is used. We had originally assumed all of our backend processing could run within Lambda's free tier limits, but some tasks needed more time or resources than Lambda allowed. When we noticed Fargate costs adding up, we looked at whether it was worth spending the time to rewrite those tasks to fit inside Lambda. Given how far along we were in the project, we decided it was more practical to accept the Fargate cost rather than slow down development to do a refactor. The total cost was still low, but it was a useful lesson in how assumptions made early in a project can affect the budget later on.

Costs for the domain name and app deployment were planned ahead of time and came in close

to what we expected. We used our sponsor's personal server to host the frontend, which kept those costs very low.

Overall, the project showed that it is possible to build and deploy a complete cloud application for very little money if you make thoughtful choices about which services to use and how to use them.

Team Members' Approach to Life-Long Learning

Clutter pushed each team member to learn new skills that none of us had much experience with before the project started. The biggest learning challenge on the backend was Go. We chose Go for our Lambda functions because it starts up faster than Python and handles multiple tasks at the same time more efficiently. Since none of us had used Go in a real project before, we had to learn it while building. The approach that worked best was to start with Go's official beginner guide and documentation, then immediately try to build something small and real, like a basic web handler or a simple data structure, before moving on to more complex parts of the system. Learning by building something real was much more effective than reading through tutorials without applying them.

Terraform was the second big learning area. Writing Terraform is different from writing regular application code. Instead of telling the computer what to do step by step, you describe what resources you want to exist and Terraform figures out how to create them. Errors also look different - they often come back as responses from the AWS API rather than simple compiler messages. The team got better at Terraform by working with real AWS accounts and real deployments rather than using fake environments. This was sometimes frustrating because mistakes cost time and occasionally money, but it led to a much deeper understanding of how everything worked. Our Lambda benchmarking experiment, where we compared Python, Go, and Java runtimes on different hardware, also helped us practice using AWS tools like CloudWatch and analyzing real performance data.

The third area was AWS deployment in general. Setting up API Gateway, writing IAM permission policies, and reading CloudWatch logs were all skills we developed from scratch during the project. We regularly referenced AWS's official documentation and best practice guides, which helped us make better decisions and understand not just how to do something but why.

The project also gave the team a hands-on lesson in database design. We started with a NoSQL database, which was a new area for most of the team. Working with non-relational data structures proved harder than expected, and it slowed down development as we struggled to model our data effectively. This pushed us to do a lot of research into how NoSQL databases work and when they are the right tool to use. Ultimately, we recognized that our data had clear relationships that were

much better suited to a relational model, and we made the decision to switch to PostgreSQL. The refactor was time-consuming, but once it was done the team moved much faster because we were working with a technology we understood well. This experience taught us something important: knowing when to change course and admit that an early technical decision was not the right one is a valuable skill, and it is something experienced engineers have to do regularly.

Looking back, this project helped the team build real confidence with cloud tools that are widely used in the industry. Go, Terraform, and AWS are all in high demand in software and DevOps roles, and having hands-on experience building and deploying a real application with them puts each team member in a stronger position when entering the job market. These are not just resume bullet points either, because we ran into real problems and had to work through them, the knowledge is practical rather than just theoretical. More importantly, the project taught us how to identify what we do not know, find reliable resources to fill that gap, and apply new skills quickly under real deadlines. That process of self-directed learning is something every engineer has to do continuously throughout their career as tools and technologies keep changing. Going through it in a structured project setting, where there were real consequences for falling behind, gave each of us a clearer sense of how to approach learning in a professional environment going forward.

Additional Material

AI Use Statement

For the following report, we used AI, that is, ChatGPT from OpenAI and Claude in general away to review the requirements for each section create report outlines, review grammar and spelling mistakes as well as making sure each section followed the rubric. No-sponsor confidential data was used in any of these tools and all written sections were verified by all team members

Full MIT License Text

MIT License

Copyright (c) 2026 Alberta 2514640 Inc.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files ("Clutter"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of Clutter, and to permit persons to whom Clutter is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.